BTL-36 – SmartBuilding – Bảng điều khiển năng lượng và điều phối tiết kiệm

> **Học phần:** PHÁT TRIỂN ỨNG DỤNG WEB CƠ BẢN – CSE122
> **Nhóm thực hiện:** 3 sinh viên
> **Loại sản phẩm:** Sản phẩm Web Frontend (nguyên mẫu) có nhiều vai trò, CRUD, responsive, tương tác JavaScript và trải nghiệm AI
> **Nguyên tắc minh chứng:** Mỗi giao diện thực tế đã được duyệt trong bảng kiểm kê màn hình = 1 video OBS có mặt sinh viên + commit/PR GitHub tương ứng.

---

1. Lĩnh vực

Smart City / Smart Building & Energy

2. Tóm tắt sản phẩm

SmartBuilding – Bảng điều khiển năng lượng và điều phối tiết kiệm là một sản phẩm Frontend giải quyết bài toán thực tế trong lĩnh vực **Smart City / Smart Building & Energy**, được thiết kế như một mini-product có nhiều role, nhiều luồng nghiệp vụ và các tính năng AI có thể mô phỏng ngay ở phía Frontend.

3. Ngữ cảnh

Tòa nhà có nhiều khu vực tiêu thụ điện khác nhau nhưng người vận hành khó biến dữ liệu đồng hồ thành cảnh báo và hành động.

4. Phát biểu vấn đề

- **P1.** Dữ liệu khó đọc với người không chuyên.
- **P2.** Khó xác định khu vực bất thường.
- **P3.** Hành động tiết kiệm chưa thành task.
- **P4.** Người dùng ít thấy tác động hành vi.

5. Mục tiêu

- **O1.** Biến bài toán thực tế thành một sản phẩm Frontend có hành trình người dùng rõ ràng.
- **O2.** Thiết kế đầy đủ giao diện cho từng vai trò, bảo đảm mỗi vai trò có ít nhất 3 màn hình.
- **O3.** Thể hiện các thao tác CRUD hoặc trạng thái nghiệp vụ phù hợp thay vì CRUD hình thức.
- **O4.** Sử dụng JavaScript/DOM cho tìm kiếm, lọc, kiểm tra hợp lệ, cửa sổ bật, thẻ, trạng thái và kết xuất dữ liệu.
- **O5.** Sử dụng JSON giả lập / LocalStorage / MockAPI / API công khai khi phù hợp.
- **O6.** Thiết kế ít nhất 3 trải nghiệm AI có thể mô phỏng được ở Frontend.
- **O7.** Tổ chức làm việc nhóm bằng Trello/Jira/Notion và Git/GitHub theo nhánh + yêu cầu hợp nhất.

6. Các vai trò

Vai trò	Trách nhiệm chính
Người sử dụng tòa nhà	Xem tiêu thụ và tham gia challenge
Nhân viên vận hành	Theo dõi zone và xử lý alert
Quản lý cơ sở	Theo dõi kpi và báo cáo
Quản trị viên	Zone, sensor mock, user

7. Hành trình người dùng tổng quát

```
Khám phá / đăng nhập
↓
Thực hiện nghiệp vụ chính theo vai trò
↓
Xem trạng thái / dữ liệu / phản hồi
↓
AI hỗ trợ phân tích hoặc gợi ý
↓
Người dùng Chấp nhận / Sửa / Từ chối / Lưu
↓
Vai trò vận hành duyệt / xử lý
↓
Bảng điều khiển / báo cáo / hoàn tất
```

NGUYÊN TẮC PHẠM VI – BẮT BUỘC ÁP DỤNG

> **Con số "tối thiểu 3 giao diện cho mỗi vai trò" chỉ là NGƯỠNG TỐI THIỂU để ngăn nhóm cố tình làm ít. Đây KHÔNG phải mục tiêu số lượng.**

Sinh viên phải phân tích đầy đủ:

```
Ngữ cảnh
→ Vấn đề
→ Vai trò
→ Mục tiêu người dùng
→ Nhiệm vụ người dùng
→ Luồng người dùng
→ Bảng kiểm kê màn hình
→ Bao phủ trạng thái
→ Triển khai
```

Và xây dựng **toàn bộ các màn hình cần thiết** để các luồng nghiệp vụ chính có thể được mô phỏng đầy đủ từ đầu đến cuối.

Một vai trò có thể cần 3, 5, 7 hoặc nhiều hơn màn hình tùy bản chất nghiệp vụ. Không được cắt bỏ màn hình cần thiết chỉ vì đã đạt số lượng tối thiểu; đồng thời không được tách một chức năng đơn giản thành nhiều trang vô nghĩa chỉ để tăng số lượng.

Tiêu chí đánh giá: ĐỘ ĐẦY ĐỦ CỦA NGHIỆP VỤ > SỐ LƯỢNG GIAO DIỆN.

Bảng kiểm kê màn hình bắt buộc trước khi viết mã

Nhóm phải lập và được duyệt bảng:

Vai trò	Mục tiêu người dùng	Nhiệm vụ người dùng	Màn hình	Tệp	CRUD/Trạng thái	AI	Người phụ trách

Giảng viên duyệt **bảng kiểm kê màn hình + luồng người dùng + Figma/Canva** trước khi nhóm triển khai code chính thức.

Bao phủ trạng thái bắt buộc

Mỗi chức năng quan trọng phải xem xét các trạng thái phù hợp:

- Bình thường
- Đang tải
- Rỗng
- Thành công
- Lỗi
- Vô hiệu hóa
- Đang chờ
- Bị từ chối
- Hoàn thành
- Đã hủy/Đã lưu trữ khi có nghiệp vụ tương ứng

Không phải trang nào cũng cần đủ mọi trạng thái, nhưng sinh viên phải chứng minh đã phân tích trạng thái phù hợp.

Màn hình dùng chung

Ngoài màn hình theo role, nhóm phải phân tích các màn hình dùng chung khi cần, ví dụ:

- `index.html`
- `login.html`
- `register.html`
- `forgot-password.html`
- `profile.html`
- `settings.html`
- `notifications.html`
- `403.html`
- `404.html`

Các màn hình này **không bắt buộc một cách máy móc**, nhưng phải được xem xét trong bảng kiểm kê màn hình.

Định nghĩa hoàn thành cho mỗi màn hình

Một màn hình chỉ được tính là **DONE** khi các mục phù hợp đã hoàn thành:

- [ ] Bản mô phỏng Figma/Canva
- [ ] HTML ngữ nghĩa
- [ ] CSS hoàn chỉnh
- [ ] Responsive
- [ ] Tương tác JavaScript
- [ ] Kiểm tra hợp lệ nếu có biểu mẫu
- [ ] Dữ liệu giả lập/API nếu cần
- [ ] Trạng thái rỗng
- [ ] Trạng thái đang tải/lỗi khi phù hợp
- [ ] Khả năng tiếp cận cơ bản
- [ ] Yêu cầu hợp nhất đã được duyệt
- [ ] Hợp nhất vào `dev`
- [ ] Video OBS
- [ ] README/bảng kiểm kê màn hình đã cập nhật

Lát cắt dọc bắt buộc cho từng sinh viên

Mỗi sinh viên phải tự hoàn thành ít nhất một luồng end-to-end:

```
Bản mô phỏng
→ HTML
→ CSS
→ JavaScript
→ Dữ liệu
→ Responsive
→ Nhánh Git
→ Ghi nhận thay đổi
→ yêu cầu hợp nhất
→ OBS
```

Không chấp nhận cách phân công mà một sinh viên chỉ làm tài liệu/Figma, một người chỉ HTML, một người chỉ JavaScript.

Trải nghiệm khi AI thất bại

Mỗi AI feature quan trọng phải có ít nhất một trạng thái thất bại hoặc không chắc chắn, ví dụ:

- “Không đủ dữ liệu để đưa ra đề xuất.”
- “AI chưa chắc chắn về kết quả này.”
- “Không thể xử lý yêu cầu lúc này.”

Và cung cấp thao tác thích hợp:

- Chỉnh sửa
- Thử lại
- Bỏ qua
- Báo cáo
- Dùng thủ công

Bảo vệ / Chỉnh sửa ngẫu nhiên

Khi bảo vệ, giảng viên có thể yêu cầu ngẫu nhiên một thay đổi nhỏ trong 5–10 phút, ví dụ:

- thêm filter;
- đổi table thành card;
- thêm field + validation;
- xử lý API trả về rỗng;
- thay đổi layout responsive;
- thêm trạng thái mới.

Mục tiêu là kiểm tra sinh viên **có thực sự hiểu và điều khiển được sản phẩm** hay không.

---

8. Danh sách giao diện nền tảng bắt buộc để khởi tạo bảng kiểm kê màn hình

Người sử dụng tòa nhà

#	Tệp	Giao diện	CRUD	Chức năng chính
1	`occupant-dashboard.html`	Occupant Dashboard	R	Xem tiêu thụ và tham gia challenge.
2	`occupant-energy-challenges.html`	Energy Challenges	C/R/U	Xem tiêu thụ và tham gia challenge.
3	`occupant-comfort-feedback.html`	Comfort Feedback	C/R/U/D	Xem tiêu thụ và tham gia challenge.

Nhân viên vận hành

#	Tệp	Giao diện	CRUD	Chức năng chính
4	`operator-energy-monitor.html`	Energy Monitor	R	Theo dõi zone và xử lý alert.
5	`operator-alert-center.html`	Alert Center	C/R/U	Theo dõi zone và xử lý alert.
6	`operator-saving-actions.html`	Saving Actions	C/R/U/D	Theo dõi zone và xử lý alert.

Quản lý cơ sở

#	Tệp	Giao diện	CRUD	Chức năng chính
7	`manager-energy-kpis.html`	Energy Kpis	R	Theo dõi kpi và báo cáo.
8	`manager-zone-comparison.html`	Zone Comparison	C/R/U	Theo dõi kpi và báo cáo.
9	`manager-monthly-report.html`	Monthly Report	C/R/U/D	Theo dõi kpi và báo cáo.

Quản trị viên

#	Tệp	Giao diện	CRUD	Chức năng chính
10	`admin-zone-management.html`	Zone Management	R	Zone, sensor mock, user.
11	`admin-sensor-management.html`	Sensor Management	C/R/U	Zone, sensor mock, user.
12	`admin-user-management.html`	User Management	C/R/U/D	Zone, sensor mock, user.

9. Chuẩn chi tiết cho TỪNG giao diện thực tế đã được duyệt

Mỗi giao diện ở mục 8 phải được nhóm triển khai đầy đủ theo danh sách kiểm tra sau:

- **Tệp HTML:** đúng tên tệp quy định.
- **Đầu trang / Điều hướng:** nhất quán trong toàn bộ vai trò.
- **Nội dung chính:** card, list, table, form hoặc dashboard phù hợp.
- **Trạng thái giao diện:** đang tải, rỗng, thành công, lỗi khi hợp lý.
- **Responsive:** desktop + tablet/mobile.
- **JavaScript:** tối thiểu một tương tác có ý nghĩa.
- **Kiểm tra hợp lệ:** với mọi biểu mẫu.
- **Dữ liệu giả lập:** không viết cứng rải rác; ưu tiên JSON/mô-đun dữ liệu.
- **Giao diện AI:** nếu trang có AI phải có Nhập → Đang xử lý → Kết quả → Giải thích → Người dùng kiểm soát.

Quy tắc CRUD

- `C` = Tạo
- `R` = Đọc
- `U` = Sửa
- `D` = Xóa
- Với dữ liệu nghiệp vụ, có thể thay Xóa bằng **Hủy / Lưu trữ / Vô hiệu hóa / Đóng** khi hợp lý.

---

10. Ma trận thực thể & CRUD gợi ý

Thực thể	Tạo	Đọc	Sửa	Xóa/Lưu trữ	Ghi chú
`zones`	✓	✓	✓	✓/Archive	Dùng mock data hoặc API giả lập.
`sensors`	✓	✓	✓	✓/Archive	Dùng mock data hoặc API giả lập.
`readings`	✓	✓	✓	✓/Archive	Dùng mock data hoặc API giả lập.
`alerts`	✓	✓	✓	✓/Archive	Dùng mock data hoặc API giả lập.
`actions`	✓	✓	✓	✓/Archive	Dùng mock data hoặc API giả lập.
`challenges`	✓	✓	✓	✓/Archive	Dùng mock data hoặc API giả lập.

11. Tính năng AI bắt buộc

AI-1

- Energy Anomaly Detector: phát hiện spike trên dữ liệu giả lập.
- **Trải nghiệm bắt buộc:** nhập dữ liệu → trạng thái đang xử lý → kết quả → lý do/giải thích → Chấp nhận/Sửa/Từ chối/Tạo lại/Lưu.
- **Cách mô phỏng:** JSON giả lập, JavaScript theo quy tắc, phản hồi định sẵn hoặc API LLM nếu nhóm đủ khả năng.

AI-2

- Saving Recommendation: gợi ý hành động tiết kiệm theo zone.
- **Trải nghiệm bắt buộc:** nhập dữ liệu → trạng thái đang xử lý → kết quả → lý do/giải thích → Chấp nhận/Sửa/Từ chối/Tạo lại/Lưu.
- **Cách mô phỏng:** JSON giả lập, JavaScript theo quy tắc, phản hồi định sẵn hoặc API LLM nếu nhóm đủ khả năng.

AI-3

- What-if Simulator: mô phỏng tác động nếu giảm X% tiêu thụ.
- **Trải nghiệm bắt buộc:** nhập dữ liệu → trạng thái đang xử lý → kết quả → lý do/giải thích → Chấp nhận/Sửa/Từ chối/Tạo lại/Lưu.
- **Cách mô phỏng:** JSON giả lập, JavaScript theo quy tắc, phản hồi định sẵn hoặc API LLM nếu nhóm đủ khả năng.

12. Nguyên tắc trải nghiệm AI

```
NGƯỜI DÙNG NHẬP
↓
KIỂM TRA HỢP LỆ
↓
AI ĐANG XỬ LÝ (đang tải / khung xương / tiến độ)
↓
KẾT QUẢ AI
↓
VÌ SAO CÓ KẾT QUẢ NÀY?
↓
CHẤP NHẬN / SỬA / TỪ CHỐI / TẠO LẠI
↓
LƯU VÀO TRẠNG THÁI ỨNG DỤNG
```

AI không được chỉ là một ô chat trang trí. Kết quả AI phải tác động vào luồng nghiệp vụ hoặc giúp người dùng đưa ra quyết định tốt hơn.

---

13. Dữ liệu giả lập / API

Các entity tối thiểu:

```
zones
sensors
```

```
readings
alerts
actions
challenges
```

Có thể triển khai bằng:

- file JSON cục bộ;
- LocalStorage / SessionStorage;
- JSON Server;
- MockAPI;
- public API;
- LLM API tùy chọn.

Không bắt buộc máy chủ thật.

---

14. Phân công nhóm 3 sinh viên

SV1

- Phụ trách chính: **Người sử dụng tòa nhà**.
- Đồng phụ trách một phần giao diện dùng chung.
- Chịu trách nhiệm responsive và kiểm tra giao diện của phần mình.

SV2

- Phụ trách chính: **Nhân viên vận hành**.
- Đồng phụ trách tương tác JavaScript / dữ liệu giả lập.
- Đánh giá yêu cầu hợp nhất của SV1 hoặc SV3.

SV3

- Phụ trách chính: **Quản lý cơ sở + Quản trị viên**.
- Chịu trách nhiệm tích hợp bố cục, điều hướng, bảng điều khiển/quản trị và phát hành.
- Đánh giá tính nhất quán toàn dự án.

> Nhóm có thể chia lại nhưng phải bảo đảm mỗi thành viên đều có HTML + CSS + JavaScript + Git + OBS và khối lượng công việc tương đối cân bằng.

Trách nhiệm chéo bắt buộc

Ngoài phần việc theo vai trò, mỗi sinh viên phải có ít nhất một trách nhiệm xuyên suốt:

- Hệ thống thiết kế / Điều hướng / Khả năng tiếp cận;
- JavaScript / Dữ liệu / Tương tác AI;
- Responsive / Tích hợp / Phát hành / Đảm bảo chất lượng.

Mục tiêu là tránh việc tạo ra ba “website con” rời rạc và buộc nhóm cộng tác thực sự.

---

15. Kho mã nguồn GitHub

Tên repo gợi ý:

```
cse122-smartbuilding-teamXX
```

Chiến lược nhánh

```
main
dev
feature/<feature-name>
fix/<bug-name>
docs/<document-name>
```

Ví dụ:

```
feature/occupant-occupant-dashboard
feature/operator-energy-monitor
feature/manager-energy-kpis
```

Quy trình bắt buộc

```
Công việc trên Trello
↓
Tạo nhánh
↓
Viết mã
↓
Ghi nhận thay đổi
↓
Đẩy lên máy chủ
↓
Yêu cầu hợp nhất
↓
Đánh giá chéo
↓
Merge vào dev
↓
Test tích hợp
↓
Merge main
```

Mẫu ghi nhận thay đổi (commit)

```
feat: add responsive dashboard layout
feat: render data from mock json
feat: add ai recommendation state
fix: validate empty form inputs
style: improve mobile navigation
refactor: split reusable ui modules
docs: update screen list and obs links
```

Không chấp nhận commit kiểu `update`, `done`, `final`, `fix code`.

Không đánh giá cao việc **spam commit**. Lịch sử phát triển phải thể hiện được quan hệ:

```
Công việc ↔ Nhánh ↔ Ghi nhận thay đổi ↔ Yêu cầu hợp nhất ↔ Đánh giá ↔ Màn hình ↔
OBS
```

---

16. Bảng công việc Trello / Jira / Notion

Tối thiểu:

```
TỒN ĐỌNG
↓
CẦN LÀM
↓
ĐANG LÀM
↓
CHỜ DUYỆT
↓
HOÀN THÀNH
```

Mỗi task phải có:

- mã công việc;
- giao diện/tệp;
- người thực hiện;
- hạn hoàn thành;
- nhánh;
- liên kết PR/commit;
- link video OBS sau khi hoàn thành.

Task mẫu:

- `TASK-01` – Khung dây + bản mô phỏng màn hình đầu tiên.
- `TASK-02` – HTML semantic cho vai trò 1.
- `TASK-03` – Responsive CSS.
- `TASK-04` – JavaScript tìm kiếm/lọc/biểu mẫu.

- `TASK-05` – Tích hợp dữ liệu giả lập/API.
- `TASK-06` – Nguyên mẫu trải nghiệm AI.
- `TASK-07` – Kiểm thử đa trình duyệt/di động.
- `TASK-08` – Minh chứng OBS + commit + README.

---

17. Video OBS – Minh chứng bắt buộc

Mỗi trang giao diện thực tế đã được duyệt = 1 video.

Số video OBS bằng số giao diện độc lập trong **bảng kiểm kê màn hình cuối cùng**. Không sử dụng con số 12 như một mục tiêu; nếu nghiệp vụ đầy đủ cần 16, 20, 24+ giao diện thì số video phải tương ứng.

Mỗi video phải:

1. Có mặt sinh viên thực hiện.
2. Hiển thị bản mô phỏng Figma/Canva.
3. Mở đúng tệp HTML/CSS/JS.
4. Giải thích bố cục và logic.
5. Thực hiện ít nhất một chỉnh sửa trực tiếp.
6. Chạy thử tương tác.
7. Ghi nhận thay đổi (commit) lên GitHub.
8. Nói rõ mã công việc/nhánh/commit.

Quy tắc đặt tên video

```
SV1-01-screen-name.mp4
SV1-02-screen-name.mp4
SV2-01-screen-name.mp4
SV3-01-screen-name.mp4
```

---

18. Figma / Canva

Trước khi code chính thức, nhóm phải có và được duyệt:

- sơ đồ trang;
- luồng người dùng;
- **bảng kiểm kê màn hình đầy đủ**;
- khung dây (khung dây);
- bản mô phỏng desktop;
- phiên bản responsive chính;
- hướng dẫn thành phần tối thiểu;
- trạng thái đang tải/rỗng/lỗi;
- bản mô phỏng tương tác AI.

Không được chỉ dùng ảnh AI sinh ra rồi viết mã theo ảnh mà không phân tích bố cục/thành phần.

---

19. Cấu trúc kho mã nguồn gợi ý

```
project-root/
├── README.md
├── docs/
│   ├── project-proposal.md
│   ├── roles-and-features.md
│   ├── screen-list.md
│   ├── team-assignment.md
│   └── ai-usage-report.md
├── design/
│   ├── figma-link.txt
│   └── mockups/
├── pages/
├── assets/
│   ├── images/
│   ├── icons/
│   └── data/
├── css/
│   ├── style.css
│   └── responsive.css
└── js/
    ├── main.js
    ├── api.js
    └── modules/
```

---

20. Khai báo sử dụng AI

Trong `docs/ai-usage-report.md`, nhóm bắt buộc khai báo:

- công cụ AI đã dùng;
- câu lệnh (prompt) chính;
- phần AI sinh ra;
- phần sinh viên chỉnh sửa;
- lỗi AI gặp phải;
- cách sinh viên kiểm chứng;
- điều sinh viên học được;
- tính năng nào chỉ mock, tính năng nào gọi API thật.

---

21. Phạm vi sản phẩm tối thiểu (MVP)

BẮT BUỘC PHẢI CÓ

- Đủ vai trò và **toàn bộ màn hình cần thiết theo bảng kiểm kê màn hình đã duyệt**; danh sách ở mục 8 chỉ là nền tảng ban đầu.
- Navigation xuyên suốt.
- Responsive.

- CRUD mô phỏng có ý nghĩa.
- Search/filter/form validation.
- Dữ liệu giả lập hoặc API.
- Tối thiểu 3 AI feature.
- Minh chứng Trello/Jira/Notion.
- Lịch sử Git + nhánh + PR.
- OBS cho từng trang.

NÊN CÓ

- Biểu đồ/bảng điều khiển.
- Thông báo nổi.
- Trạng thái rỗng/đang tải/lỗi.
- Thành phần và token tạo kiểu tái sử dụng.
- Chế độ tối/sáng hoặc cải thiện khả năng tiếp cận nếu phù hợp.

CÓ THÌ TỐT

- LLM API thật.
- Hoạt ảnh/tương tác vi mô.
- PWA/bộ đệm cục bộ.
- Cá nhân hóa chủ đề.
- Biểu đồ/bản đồ nâng cao.

---

22. Tiêu chí duyệt đề

Đề tài chỉ được coi là hoàn thành khi giảng viên có thể kiểm tra chuỗi:

```
Ngữ cảnh
→ Vấn đề
→ Mục tiêu
→ Vai trò
→ Màn hình
→ Bản mô phỏng
→ Công việc
→ Nhánh
→ Ghi nhận thay đổi
→ yêu cầu hợp nhất
→ OBS
→ Sản phẩm
```

Nếu không chứng minh được chuỗi này, nhóm chưa chứng minh đầy đủ quá trình học và thực hành. Việc chỉ đạt ngưỡng số lượng tối thiểu không đồng nghĩa với hoàn thành tốt BTL.

---

23. Điểm sáng tạo của đề tài

Đề tài này khác CRUD truyền thống vì nó yêu cầu:

- nhiều vai trò tương tác;
- trạng thái nghiệp vụ rõ;
- UX quyết định/điều phối;
- AI có giải thích;
- dữ liệu được thể hiện bằng bảng điều khiển, danh sách, thẻ, bộ lọc hoặc dòng thời gian;
- teamwork và bằng chứng quá trình phát triển.

---

24. Chuẩn đầu ra mong đợi

Sau BTL, sinh viên phải chứng minh có thể:

- phân tích một vấn đề thực tế;
- chia hệ thống thành vai trò và màn hình;
- thiết kế UI/UX bằng Figma/Canva;
- hiện thực HTML semantic;
- dùng CSS/Bootstrap và responsive;
- lập trình DOM/sự kiện/biểu mẫu bằng JavaScript;
- dùng JSON/API;
- mô phỏng trải nghiệm AI có ý nghĩa;
- làm việc bằng Git/GitHub;
- giao việc và review;
- giải thích lại toàn bộ sản phẩm bằng video OBS.